第十二章 代码优化

赵银亮 西安交通大学 2026

© TemplatesWise.com

知识图谱

- 控制流分析

  CFG、指令/程序入口、出口、结构化块、基本块

  succ(i)、pred(i)、path[i,j]、path(i,j)、i dom j、dom(i)、 idom(i)

  Path[i,j]

- 数据流分析

  gen[i]、kill[i]、def[i]、use[i]、in[i]、out[i]、

  数据流方程

  到达定值分析、UD链

  活跃变量分析、DU链

- 寄存器分配

  web、极大web、干涉图、着色算法、溢出、溢出代码改写

- 其他优化

控制流分析

- 将程序限定为函数代码块，是指令序列。
- 程序$P=[1,^{\#}P]$是它的每条指令的索引值所构成的集合。
- 指令 i 的静态（直接）前驱是指令 i-1，其中 i>1。
- 指令 i 的静态（直接）后继是指令 i+1，其中 $i<^{\#}P$。
- 指令i的（动态直接）前驱集是pred(i) 。
- 指令i的（动态直接）后继集是succ(i)。
- 进一步将程序P限定为只能是结构化代码块：
  - 索引值最小的指令为其唯一入口指令，最大的为唯一出口指令
  - 入口指令无前驱，出口指令$^{\#}P$无后继。
  - 一般地，通过添加指令0，即如 LABELf 来保证入口指令满足
  - 事实上，f@label可满足，f@code不一定满足。

QTAC 的指令 i 的后继集 succ(i)

指令i	succ(i)
GOTO l	$\{l\}$
IF $q\ r\ a$ THEN l_1 ELSE l_2	$\{l_1, l_2\}$
v=CALL g, k	$\{i+1\}$
RETURN a	$\emptyset$
0	$\{1\}$
$^{\#}P$	$\emptyset$
其他指令	$\{i+1\}$

- 利用以下性质计算前驱集：
  - $j \in \text{succ}(i) \Leftrightarrow i \in \text{pred}(j)$ 或
  - $\forall\, i, j \in P \cdot j \in \text{succ}(i) \rightarrow i \in \text{pred}(j)$

控制流图

- 控制流图（CFG）可分以基本块为顶点的和以指令为顶点的两种，前者为默认CFG，后者为指令级CFG。本课程只考虑后者。

- 定义12.1 一个程序 P 的CFG 是一个有向图 $G(V, E)$，其中，$V = P \cup \{0\}$，$E = \{\langle t, h\rangle \mid t \in \text{pred}(h),\ h \in \text{succ}(t),\ t, h \in P\}$。

- 对于一个CFG的任意两个结点 i 和 j 有：
  - path$[i, j]$ 表示始端为 i，末端为 j 的所有路径构成的集合。
  - path(i, j)为真当且仅当 path$[i, j]$ 非空。
- path$[i, j] = \{(i, j) \mid \text{path}(i, j),\ i,j \in P\}$。
- Path$[i, j] = \cup\, p \in \text{path}[i, j] \cdot V(p)$
- 函数 $V(p)$ 返回路径 p 的结点集。

支配集dom(j)

- 若 $\forall\, p \in \text{path}[0, j] \cdot i \in V(p)$ 那么称 i 支配 j，俗称 i 是 j 的必经结点，记为 $i \text{ dom } j$。这里函数 $V(p)$ 返回路径 p 的结点集。
- $\text{dom}(j) = \{i \in P \mid i \text{ dom } j\}$ 被称为是 j 的支配集。
- 性质：
  - $j \in \text{dom}(j)$ 自反性。
  - $\forall\, j \in P \cdot 0 \in \text{dom}(j)$ 起点唯一性。
  - $i \in \text{dom}(j) \wedge j \in \text{dom}(k) \rightarrow i \in \text{dom}(k)$ 传递性。
  - $i \text{ dom } j$ 那么 $\text{dom}(i) \subseteq \text{dom}(j)$单调性。
- 说直接支配集idom(i)同时满足：
  - $\text{idom}(i) \in \text{dom}(i)$
  - $\text{idom}(i) \text{ dom } j \wedge j \text{ dom } i \rightarrow i = j$
  - $\text{idom}(0) = \perp$

示例程序fib@code

1. a=0
2. b=1
3. z=0
4. LABEL loop
5. IF n==z THEN end ELSE body
6. LABEL body
7. t=a+b
8. a=b
9. b=t
10. n=n-1
11. z=0
12. GOTO loop
13. LABEL end
14. RETURN a

1 dom i √

4 dom 5 √

5 dom 6 √

5 dom 11 √

11 dom 5 ×

dom(11)=[1,11] √

dom(5)=[1,5] √

数据流分析

- use[i]：结点 i 使用的变量集
- def[i]：结点 i 定值的变量集
- gen[i]：结点 i 生成的定值集
- kill[i]：结点 i 消除的定值集

数据流分析

- 变量的定值与使用

指令i	def[i]	use[i]
LABEL l	$\emptyset$	$\emptyset$
GOTO l	$\emptyset$	$\emptyset$
IF $q\ r\ x$ THEN l_1 ELSE l_2	$\emptyset$	$\{q, x\}$
PAR x	$\emptyset$	$\{x\}$
$v =$ CALL g, k	$\{v\}$	$\emptyset$
RETURN x	$\emptyset$	$\{x\}$
$z = y\ o\ x$	$\{z\}$	$\{y, x\}$
$y = M[x]$	$\{y\}$	$\{x\}$
$M[x] = y$	$\emptyset$	$\{x, y\}$
$y = x$	$\{y\}$	$\{x\}$

例：变量定值集、使用集；定值生成集、消除集

i	Ins	def[i]	use[i]	gen[i]	kill[i]
1	a=0;	{a}	∅	{d1}	∅
2	b=1;	{b}	∅	{d2}	∅
3	z=0;	{z}	∅	{d3}	∅
4	LABEL loop;	∅	∅	∅	∅
5	IF n==z...	∅	{n,z}	∅	∅
6	LABEL body;	∅	∅	∅	∅
7	t=a+b;	{t}	{a,b}	{d7}	∅
8	a=b;	{a}	{b}	{d8}	{d1}
9	b=t;	{b}	{t}	{d9}	{d2}
10	n=n-1;	{n}	{n}	{d10}	{d0}
11	z=0;	{z}	∅	{d11}	{d3}
12	GOTO loop;	∅	∅	∅	∅
13	LABEL end;	∅	∅	∅	∅
14	RETURN a;	∅	{a}	∅	∅

到达定值分析

- 到达定值分析：是一种正向（Forward）的数据流分析。它的目标是追踪变量的每个定值 d 在程序中的流动，找出在某个程序点，变量可能的定值都有哪些，即寻找 UD 链。用于常量传播、死代码删除。
- in[i]：结点 i 入口定值集
- out[i]：结点 i 出口定值集
- gen[i]：结点 i 生成的定值集
- kill[i]：结点 i 消除的定值集

- 数据流方程：
- $\text{out}[i] = \text{gen}[i] \cup (\text{in}[i] \setminus \text{kill}[i])$
- $\text{in}[i] = \cup_{j \in \text{pred}(i)} \cdot \text{out}[j]$

例: $out[i] = gen[i] \cup (in[i] \setminus kill[i])$ $in[i] = \cup_{j \in pred(i)} \cdot out[j]$

					迭代1		迭代2		迭代3	
i	use[i]	pred(i)	gen[i]	kill[i]	out[i]	in[i]	out[i]	in[i]		
1			a1		01	0	01	0		
2		1	b2		012	01	012	01		
3		2	z3		0123	012	0123	012		
4		3, 12			0123	0123	0123789AB	0123789AB		
5	n, z	4			0123	0123	0123789AB	0123789AB		
6		5			0123	0123	0123789AB	0123789AB		
7	a, b	6	t7		01237	0123	0123789AB	0123789AB		
8	b	7	a8	a1	02378	01237	023789AB	0123789AB		
9	t	8	b9	b2	03789	02378	03789AB	023789AB		
10	n	9	nA	n0	3789A	03789	3789AB	03789AB		
11		10	zB	z3	789AB	3789A	789AB	3789AB		
12		11			789AB	789AB	789AB	789AB		
13		5			0123	0123	0123789AB	0123789AB		
14	a	13			0123	0123	0123789AB	0123789AB		

定值点集、UD链

- 指令i 有关于x 的定值d，即$x \in \text{def}[i]$，就将该定值d写为d[x,i]以具体化。
- 一条指令最多产生一个定值（由IR和机器指令保证）。
- P 中所有关于x 的定值构成一个集合，用defs[x]表示。
- $\text{kill}[i] = \text{defs}[x] \backslash \{\text{d}[x,i]\}$
-
- 变量 x 在语句 j 处的 UD 链
  - $\text{UD}(j, x) = x \in \text{use}[j] \ ? \ \text{in}[j] \cap \text{defs}[x] : \emptyset$

活跃变量分析

- 活跃变量分析：是一种反向（Backward）的数据流分析。它的目标是追踪变量的“使用（Use）”，判断在某个程序点，变量的值在后续的执行路径中是否还会被读取。
- use[i]：结点 i 使用的变量集
- def[i]：结点 i 定值的变量集
- in[i]：结点 i 入口活跃变量集（Live-in[i]）
- out[i]：结点 i 出口活跃变量集（Live-out[i]）
- 数据流方程：
- $in[i] = use[i] \cup (out[i] \setminus def[i])$
- $out[i] = \cup_{j \in succ(i)} \cdot in[j]$

例：$in[i] = use[i] \cup (out[i] \setminus def[i])$　$out[i] = \cup_{j \in succ(i)} \cdot in[j]$

	注：初始化为空集			迭代1		迭代2		迭代3	
i	$succ(i)$	$use[i]$	$def[i]$	$out[i]$	$in[i]$	$out[i]$	$in[i]$	$out[i]$	$in[i]$
1	2		a	a, n	n	a, n	n	a, n	n
2	3		b	a, b, n	a, n	a, b, n	a, n	a, b, n	a, n
3	4		z	a, b, n, z	a, b, n, z	a, b, n	a, b, n	a, b, n	a, b, n
4	5			a, b, n, z	a, b, n, z	a, b, n, z	a, b, n	a, b, n, z	a, b, n
5	6, 13	n, z		a, b, n	a, b, n, z	a, b, n	a, b, n, z	a, b, n	a, b, n, z
6	7			a, b, n	a, b, n	a, b, n	a, b, n	a, b, n	a, b, n
7	8	a, b	t	b, t, n	a, b, n	b, t, n	a, b, n	b, t, n	a, b, n
8	9	b	a	t, n	b, t, n	t, a, n	b, t, n	t, a, n	b, t, n
9	10	t	b	n	t, n	a, b, n	t, a, n	a, b, n	t, a, n
10	11	n	n		n	a, b, n	a, b, n	a, b, n	a, b, n
11	12		t			a, b, n, z	a, b, n	a, b, n, z	a, b, n
12	4					a, b, n, z	a, b, n, z	a, b, n, z	a, b, n, z
13	14			a	a	a	a	a	a
14		a			a		a		a

定义-使用网/变量网/web

$j \in \text{web}(x, i) \Leftrightarrow j = i \vee$

$i \text{ dom } j \wedge$

$(x \in \text{out}[j] \vee x \in \text{use}[j]) \wedge$

$\forall\, k \in \text{Path}[i, j] \setminus \{i\} \cdot x \notin \text{def}[k]$

;$x \in \text{def}[i]$, $i \in \text{web}(x, i)$

;$\forall\, p \in \text{path}[0, j] \cdot i \in V(p)$

;$\text{in}[i] = \text{use}[i] \cup (\text{out}[i] \setminus \text{def}[i])$

;$\text{out}[i] = \bigcup j \in \text{succ}(i) \cdot \text{in}[j]$

;并且从 i 到 j 的所有路径中，除 i 以外没有任何结点重新定义 x。

例：求web(*x*, *i*)

- web(a,1)={1,2,3,4,5,6,7,13,14}
- web(a,8)={8,9,10,11,12,4,5,6,7,13,14}
- web(b,2)={2,3,4,5,6,7,8}
- web(b,9)={9,10,11,12,4,5,6,7}
- web(z,3)={3,4,5}
- web(z,11)={11,12,4,5}
- web(t,7)={7,8,9}
- web(n,10)={10,11,12,4,5}
- web(n,0)={1,2,3,4,5}

1. a=0
2. b=1
3. z=0
4. LABEL loop
5. IF n==z THEN end ELSE body
6. LABEL body
7. t=a+b
8. a=b
9. b=t
10. n=n-1
11. z=0
12. GOTO loop
13. LABEL end
14. RETURN a

极大web

- 合并web：如果$\text{web}(x,i) \cap \text{web}(x,j) \neq \emptyset$ 则二者可合并。

- 变量的web集：$\text{web}(x) = \{\text{web}(x,i) \mid x \in \text{def}[i]\}$

- 变量的极大web集：

$\text{maxweb}(x) = \{\bigcup X \in B \cdot X \mid B \in \text{web}(x)/\mathcal{R}(x)\}$，其中，等价关系 $\mathcal{R}(x) = \{(w_1, w_2) \mid w_1, w_2 \in \text{web}(x), w_1 \cap w_2 \neq \emptyset\}$

- 程序P 的极大web集：$\text{maxweb}(P)$为其所有变量的极大web集的并集。

干涉图

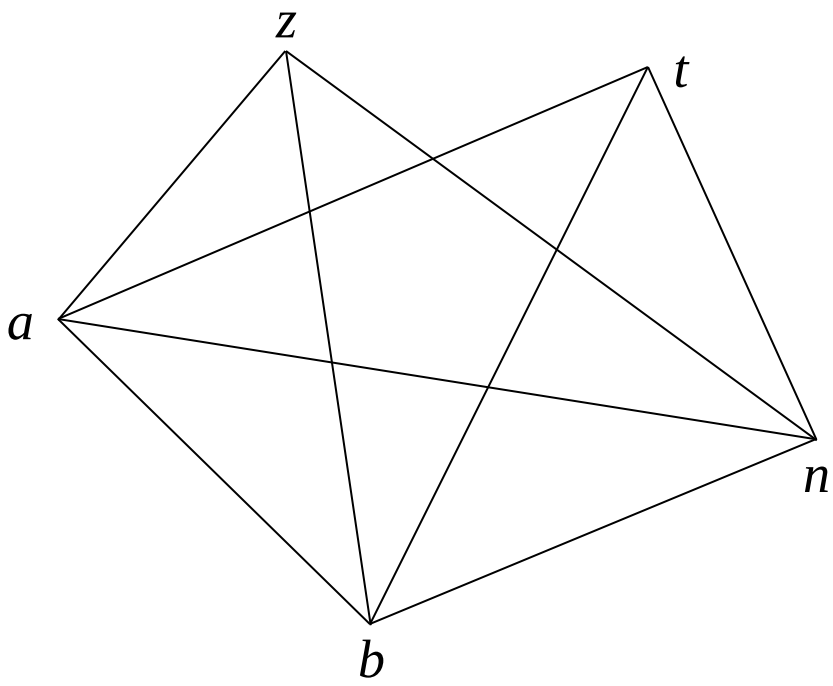
- 程序P 的干涉图$G(V, E)$，其中，
  - V=maxweb(P)
  - $(w_1, w_2) \in E \Leftrightarrow w_1 \cap w_2 \neq \emptyset$

- 结点规则：一个变量最终可能产生一个或多个互不相交的极大 Web。在构建干涉图时，每一个极大 Web 都是图中的一个独立结点。
- 连边规则：如果两个极大 Web（可以是不同变量，也可以是同一变量的不同极大 Web）的活跃区间有重叠，它们之间就会有一条干涉边。

干涉图

- 由于每个变量的极大web集都是单元素的，所以图中就用变量标记它的那个极大web。

- web(a,1)={1,2,3,4,5,6,7,13,14}
- web(a,8)={8,9,10,11,12,4,5,6,7,13,14}
- web(b,2)={2,3,4,5,6,7,8}
- web(b,9)={9,10,11,12,4,5,6,7}
- web(z,3)={3,4,5}
- web(z,11)={11,12,4,5}
- web(t,7)={7,8,9}
- web(n,10)={10,11,12,4,5}
- web(n,0)={1,2,3,4,5}



基于图着色的寄存器分配

- 在干涉图中，若两个变量对应的结点之间无边连接（即互不干涉），则它们可共享寄存器。因此，寄存器分配问题可转化为图着色问题：为干涉图中的每个结点分配一个“颜色”（对应寄存器编号），满足以下两个条件：
- 1. 相邻结点（有边连接）的颜色不同；
- 2. 使用的不同颜色数量不超过可用寄存器数量。

算法12.1 乐观着色启发式寄存器分配算法。

输入： 干涉图、颜色数N

输出： 结点与其颜色标记/待溢出标记

（1）初始化：创建空栈（用于暂存结点）。

（2）简化步骤（Simplify Phase）：

（2.1）若存在度数小于N的结点，将该结点及其相邻结点列表压入栈，并从图中移除该结点及所有关联边；

（2.2）若所有结点度数均不小于N，任选一个结点，执行上述操作；

（2.3）若图中仍有结点，重复简化步骤；否则进入（3）选择步骤。

（3）选择步骤（Select Phase）：

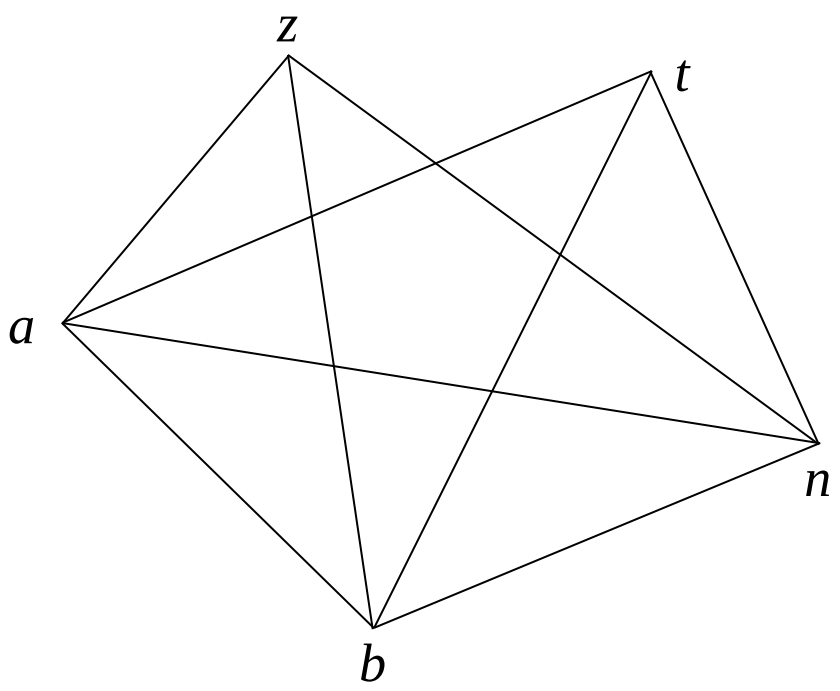
（3.1）从栈中弹出一个结点及其相邻结点列表；

（3.2）若存在一种颜色，与该结点所有已着色相邻结点的颜色不同，则为其分配该颜色；

（3.3）若不存在这样的颜色，着色失败，标记该结点为待溢出；

（3.4）若栈中仍有结点，重复（3）选择步骤；否则算法结束。

算法应用示例



结点栈	邻居结点	颜色
z	a, b, n	2
a	b, t, n	4
b	t, n	3
t	n	2
n		1

溢出

- 若算法12.1的选择步骤中无法为某个结点分配颜色，说明无法将所有变量全程驻留于寄存器中，需选择部分变量让其驻留于内存（仅在使用时临时加载到寄存器），这一过程称为溢出（spilling）。待溢出的候选变量通常是如选择步骤中无法着色的结点对应的变量。
- 标记待溢出变量后，继续对栈中剩余结点执行选择步骤（为其他结点着色时忽略已溢出变量）。算法结束时，可能有多个变量被标记为溢出。

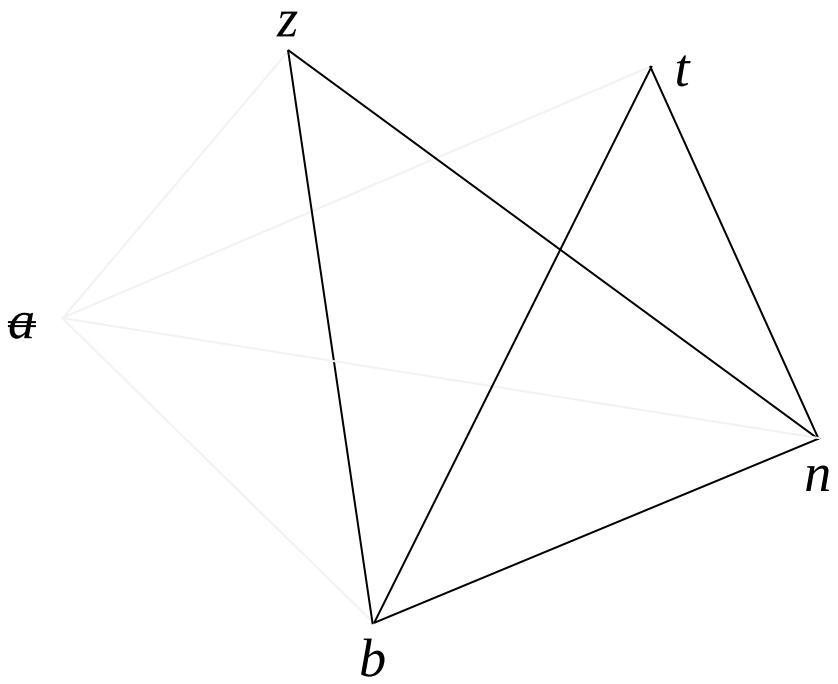
溢出变量的程序改写

- 对每个溢出变量 x，需按以下步骤修改程序，确保其值存储在内存中：
- 1. 为 x 分配一个内存地址 l_x，用于存储 x 的值。
- 2. 对所有读或写 x 的指令 i，将指令内的 x 重命名为 xi（局部临时变量）。
- 3. 在读取 xi 的指令 i 之前点，插入指令 $xi = M[\,l_x\,]$（从内存加载 x 的值到局部临时变量）。
- 4. 在赋值 xi 的指令 i 之后点，插入指令 $M[l_x] = xi$（将临时变量的值写回内存）。
- 5. 若 x 是输入参数，在函数起始处添加指令 $M[l_x] = x$（将参数 x 的值存入内存，此处使用 x 的原始名称）。

溢出变量的程序改写

- 需要注意的是，对于读后写x的指令i，需要分别在其前点插入指令$xi = M[l_x]$，同时在其后点插入指令$M[l_x] = xi$。程序改写后，需重新执行寄存器分配（包括重新进行活跃变量分析）。这是因为新增了临时变量xi，且x的活跃性已改变。可通过仅对受影响变量（xi和x）重新执行活跃变量分析，而其他变量的活跃性不变，以此保持效率。

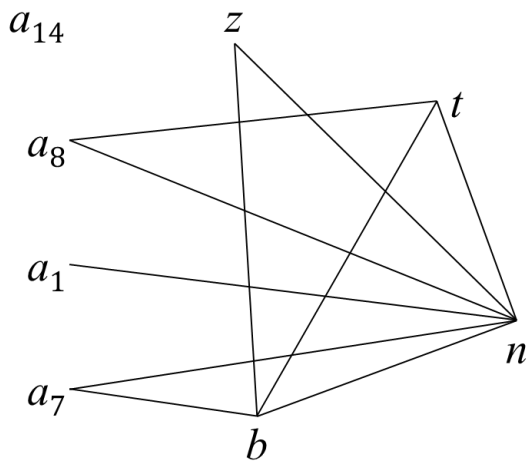
算法应用示例



结点栈	邻居结点	颜色
n		1
t	n	2
b	t, n	3
~~a~~	~~b, t, n~~	溢出
z	~~a~~, b, n	2

再次进行寄存器分配

1. $a_1 = 0$;
   $M[l_a]=a_1$;
2. $b = 1$;
3. $z = 0$;
4. LABEL loop;
5. IF $n == z$ THEN end ELSE body;
6. LABEL body;
7. $a_7=M[l_a]$;
   $t = a_7 + b$;
8. $a_8 = b$;
   $M[l_a]=a_8$;
9. $b = t$;
10. $n = n - 1$;
11. $z = 0$;
12. GOTO loop;
13. LABEL end;
    $a_{14}=M[l_a]$;
    RETURN a_{14};



结点栈	邻居结点	颜色
n		1
t	n	2
a_8	t, n	3
b	t, n	3
a_7	b, n	2
z	b, n	2
a_1	n	2
a_{14}		1

共同构建引用-定值链（UD链 / DU链）

- 到达定值分析帮助编译器建立 UD链（Use-Definition Chain）：当程序在某处使用变量 x 时，UD链能告诉编译器 x 的值可能是在哪里被赋值的。
- 活跃变量分析帮助编译器建立 DU链（Definition-Use Chain）：当程序在某处对变量 x 赋值时，DU链能告诉编译器这个值可能在后续的哪些地方被使用。
- 这两种链是编译器进行常量传播、公共子表达式消除等高级优化的基础数据支撑

死代码消除（Dead Code Elimination）

- 要删除一段无用的赋值语句（如 $x = y + z$），编译器需要同时满足两个条件：
- 利用活跃变量分析：确认变量 x 在赋值后，直到程序结束都不再被使用（即不活跃）。
- 利用到达定值分析：确认该赋值语句确实到达了当前点，且没有被其他赋值覆盖。
- 只有两者结合，编译器才能安全地判定这是一条“死代码”并将其删除。

共同指导寄存器分配

- 活跃变量分析决定了变量的生命周期（何时需要分配寄存器，何时可以释放）；而到达定值分析则帮助识别出那些可以合并或优化的变量版本，两者共同为寄存器分配器提供精确的静态信息

消除冗余传送指令

- 若变量x和y被分配到同一个寄存器，那么形如x＝y的赋值指令将失去实际作用，可从代码中删除。
- 偏向着色（Biased Colouring）消除形如x＝y的冗余传送指令：对寄存器分配器而言，若为y分配颜色时，x已完成着色，那么只要x的颜色未被y的任何相邻结点占用，就为y分配与x相同的颜色
- 寄存器合并（Coalescing）。在着色前，若x和y不存在干涉，将二者对应的结点合并为一个结点
- 合并后，x和y自然共享同一寄存器，x＝y指令就变得多余了，可直接消除。
- 与合并相反的技术是活跃区间拆分（Live-Range Splitting），同样可用于减少溢出。其思路是无需溢出整个变量，而是将变量的活跃区间（从首次使用到最后一次使用的代码范围）进行拆分，具体是为变量在各拆分区间的出现分配不同名称，并在必要时插入传送指令连接这些名称。